TOPIC :-

# Indexes Management

⇒ Indexes are a crucial component for optimizing performance in oracle database.

⇒ They function like an index in a book, allowing for faster retrival of specific data. it is a memory object.

What are indexes?

- Optional structures associated with tables and clusters.
- Speed up execution of SQL statements by providing a faster access path to data.
- Allows faster access to rows in a table when a small subset of the rows will be retrieved from the table.
- An index stores the value of the column or columns being indexed, along with the physical rowid of the row containing the indexed value.
- indexes are logically and physically independent of the data of associate table.
- Indexes are created on a single column or multiple columns.
- Index entries are stored in a B-tree structures so that traversing the index to find the key value of the row uses very few i/o operations.
- Primarily means of reducing I/O on disk

## Types of indexes

⇒ There are the following types:

- B-Tree indexes (balanced trees)
  - These are the default type and work well for various queries.
  - Data is organized in a tree-like structure for efficient searching.
  - Ideal for selective queries.
- Bit-map indexes:
  - Designed for data warehouses with large datasets and many filtering conditions. Appropriate on low distinct cardinality
  - Use bitmaps to represent sets of rows that meet certain criteria.
  - Efficient for queries with multiple WHERE clause conditions, especially when filtering on low cardinality columns.

when creating an index root block is created then branch blocks & finally leaf blocks and each branch holds the range of data its leaf blocks

(RowID and index key value)

- useful on columns that appear in where clause

- it stores one string of bits for each possible value of the column being indexed.

- **Unique indexes:**
  - Ensures each indexed value appears only once in the table. Enforced by the database.
  - it is most common form of B-Tree index.

  it is often used to enforce the primary key constraint of a table.
  Ensures no duplicate value exist.

- **Non-Unique indexes:**
  - Allows for duplicate values in the indexed column(s). Useful for speeding up searches based on those columns.

- Helps speed access to a table without enforcing uniqueness.

## Additional Index Types

**Function based index:**

- Store precomputed values of functions applied to indexed columns.
- Useful for speeding up queries involving functions on those columns.
- it is same like to a standard B-index, except that a transformation of a column or columns, declared as an expression, is stored in the index instead of the columns themselves.
- Useful in cases where names and addresses might be stored in the database as mixed case.

**Reverse key index:**

- Order the index entries in reverse order, beneficial for specific situations like sorting results in descending order. useful when created or column that contain sequential data
- Typically used in OLTP environment.
- In reverse key index, all the bytes in each column's key value of the index are reversed.

**Domain index:**

- Used with oracle cartridges for specific data types.

# Management on indexes

## 1. Creating indexes:

[Note: syntaxes on registers]

- Explicit index creation:

Syntax:

⇒ CREATE INDEX idx_customer_name
ON customers;

- Index with constraints:

Syntax:

⇒ ALTER TABLE orders
ADD CONSTRAINTS PK_order
PRIMARY KEY;

## 2. Monitoring indexes:

Identifying unused indexes:-

Syntax:

⇒ SELECT owner, index_name, last_analyzed
FROM user_indexes
WHERE last_analyzed IS NULL
OR
SYSDATE - last analyzed

- Monitoring fragmentation:
- Over time, as data is inserted deleted or updated indexes can become fragmented.
- This reduces their effectiveness in speeding up queries.
- monitoring fragmentation levels with tools like DBMS_STATS helps determine if rebuilding is necessary.

## 3. Altering indexes:

Modifying storage characteristics:

⇒ By adjusting storage parameters like tablespace placement or setting fill factor to optimize index usage and storage efficiently

Syntax:

⇒ ALTER INDEX idx_customer_name
SET TABLESPACE idx_tbspace;

Rebuilding indexes:

Syntax:

⇒ ALTER INDEX idx_product REBUILD;

Making index usable/unusable:

Syntax:

⇒ ALTER INDEX idx_oxcless DISABLE;

Syntax: ——— usable

⇒ ALTER INDEX idx_oxcless ENABLE;

Making index visible/ unvisible:

Syntax:

⇒ ALTER INDEX idx_customer VISIBLE FALSE;

ALTER INDEX idx_customer VISIBLE TRUE;

## 4. Dropping indexes:

Syntax:

⇒ DROP INDEX idx_old_data;